



Chapter 3: Unsupervised Learning

Ph.D. Nguyen Thi Khanh Tien
tienntk@ut.edu.vn

Introduction to Unsupervised Learning

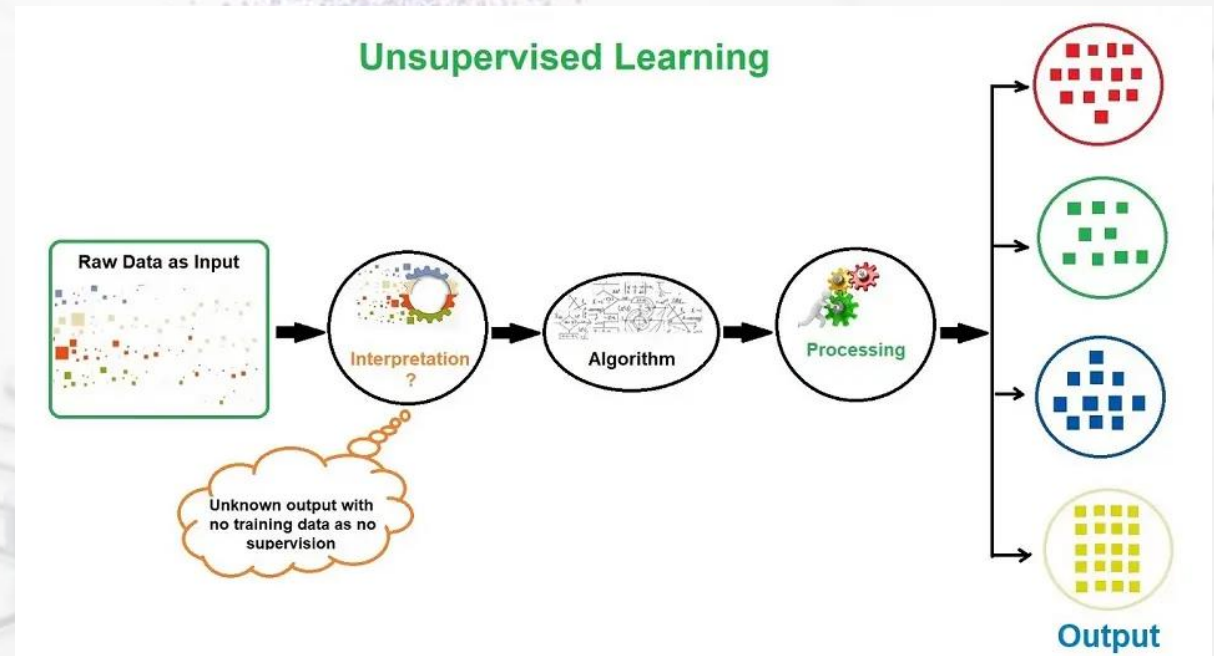
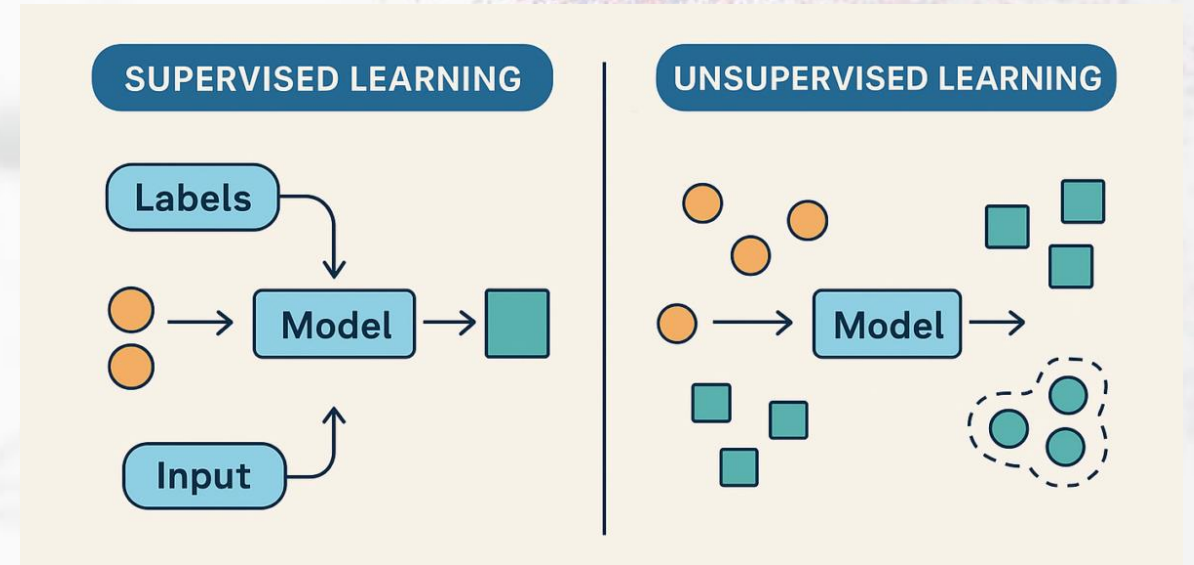
Unsupervised learning is a machine learning technique where the algorithm learns patterns and structures within data without explicit guidance or labels. Unlike supervised learning, which requires labeled data to train a model, unsupervised learning focuses on discovering hidden insights and relationships within the data itself.

Key Characteristics of Unsupervised Learning

- ❖ **No Labels:** The algorithm is not provided with predefined target variables or classes.
- ❖ **Pattern Discovery:** The goal is to identify patterns, clusters, or anomalies within the data.
- ❖ **Exploratory Analysis:** Unsupervised learning is often used for exploratory data analysis to gain a better understanding of the underlying structure.

Applications of Unsupervised Learning

- ❖ **Customer Segmentation:** Grouping customers based on their behavior or demographics.
- ❖ **Image and Video Analysis:** Identifying objects, scenes, or anomalies in visual data.
- ❖ **Fraud Detection:** Detecting unusual patterns in financial transactions.
- ❖ **Recommendation Systems:** Suggesting items or content based on user preferences.
- ❖ **Bioinformatics:** Analyzing gene expression patterns or protein structures.



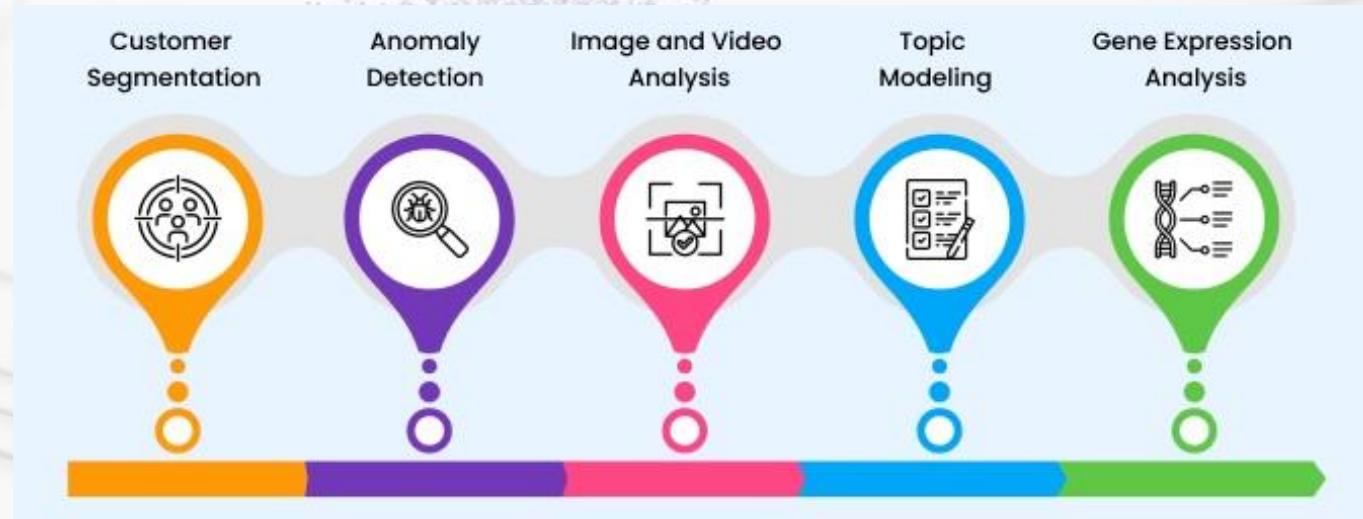
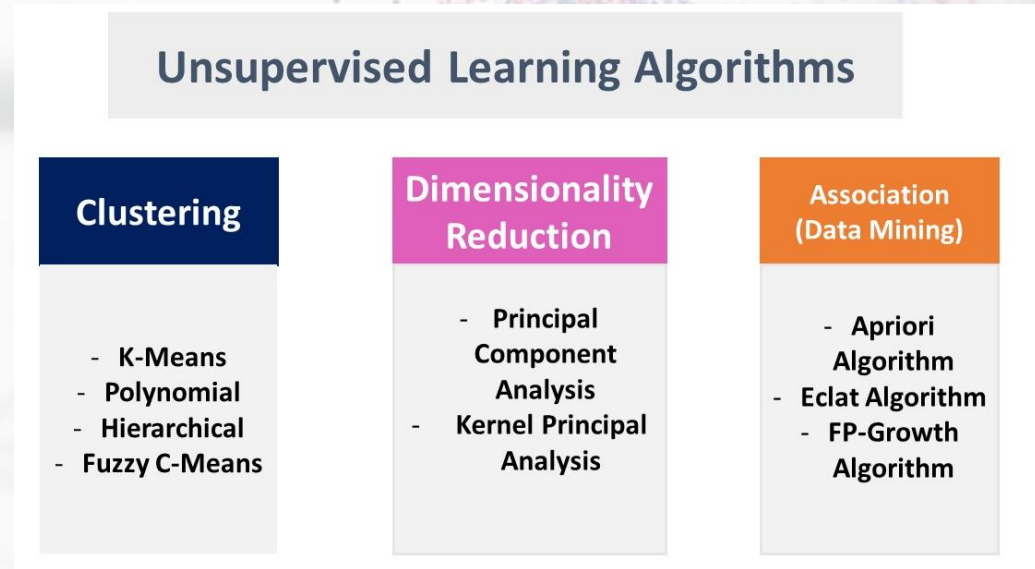
Introduction to Unsupervised Learning

Common Unsupervised Learning Techniques

- **Clustering:** Grouping similar data points together.
 - Examples: K-means, hierarchical clustering, DBSCAN
- **Dimensionality Reduction:** Reducing the number of features while preserving essential information.
 - Examples: Principal Component Analysis (PCA), t-SNE
- **Anomaly Detection:** Identifying unusual data points that deviate from the norm.
 - Examples: Isolation Forest, One-Class SVM
- **Association Rule Mining:** Discovering relationships between items in a dataset.
 - Example: Apriori algorithm

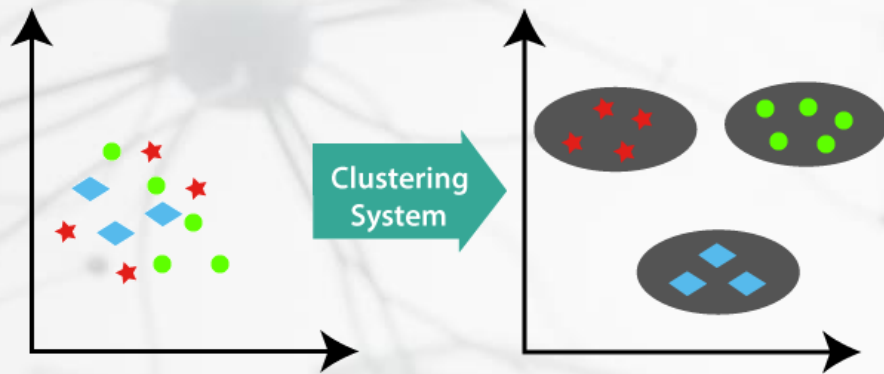
Challenges and Considerations

- **Evaluation:** Assessing the quality of unsupervised learning results can be challenging due to the lack of ground truth.
- **Interpretation:** Understanding the patterns discovered by unsupervised algorithms can require domain expertise.
- **Scalability:** Some unsupervised algorithms may struggle with large datasets.



Introduction to Clustering

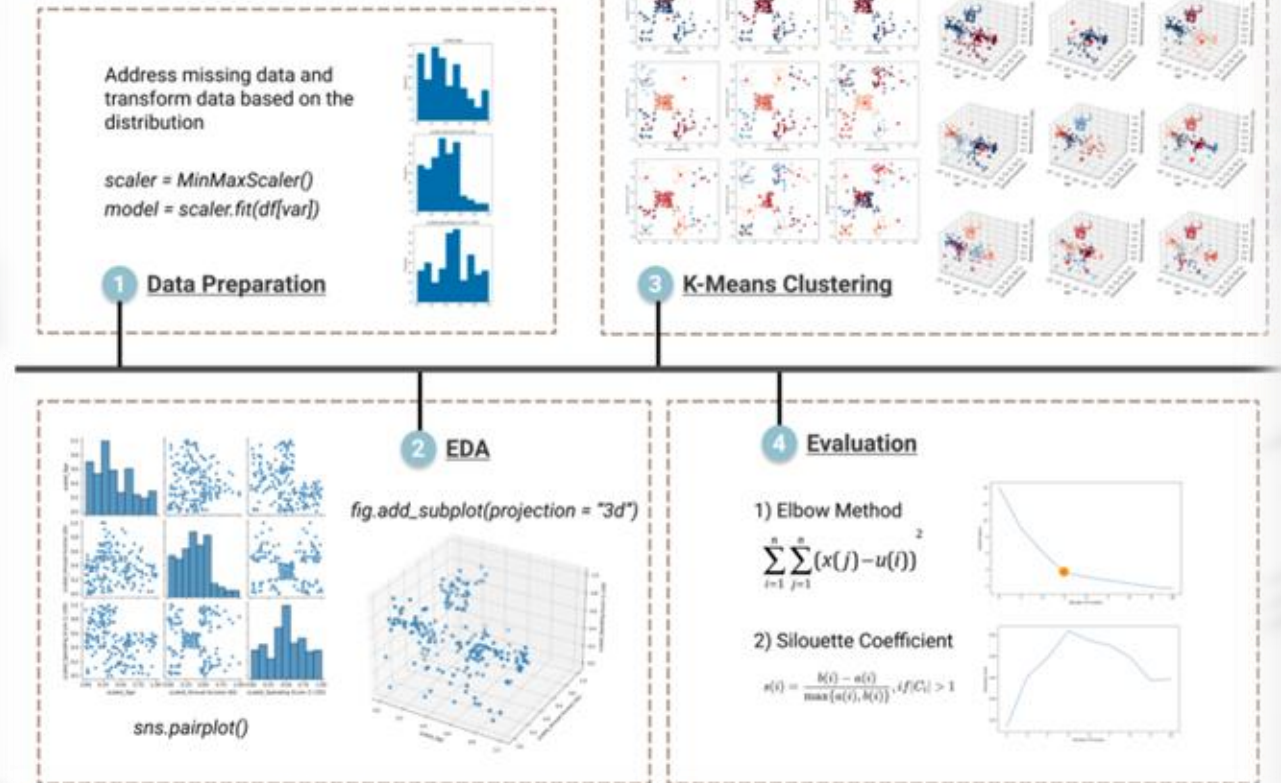
Clustering is a machine learning technique used to group similar data points together. It's a fundamental unsupervised learning task where the algorithm identifies natural patterns or structures within the data without any prior labeling.



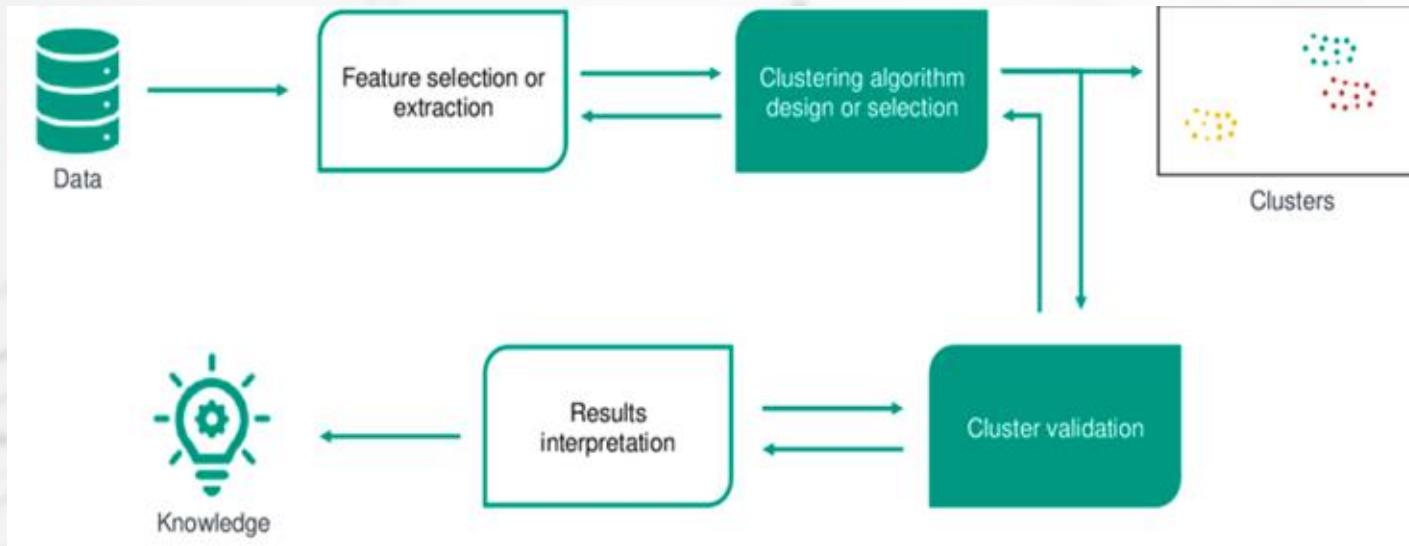
Applications of Clustering

- Customer Segmentation: Grouping customers based on their demographics, purchasing behavior, or preferences.
- Image Segmentation: Dividing an image into distinct regions based on color, texture, or shape.
- Anomaly Detection: Identifying unusual data points that deviate from the norm.
- Social Network Analysis: Discovering communities or groups within a network.
- Bioinformatics: Analyzing gene expression patterns or protein structures.

CLUSTERING ANALYSIS & CUSTOMER SEGMENTATION.



Clustering pipeline



1. Data Preparation

- **Collect** relevant data.
- **Clean** data: Remove noise, outliers, and missing values.
- **Engineer** features for better clustering.
- **Normalize** or **standardize** features to a common range.

2. Distance Calculation

- **Choose** a distance metric (e.g., Euclidean, Manhattan, Cosine, Hamming).
- **Calculate** distances between all data points.

3. Cluster Formation

- **Initialize** clusters: Choose k and randomly select k initial centers.
- **Assign** data points to nearest centers.
- **Update** cluster centers based on assigned points.

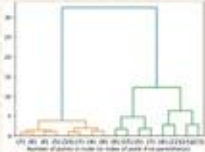
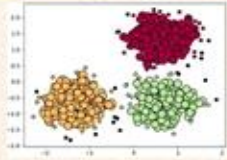
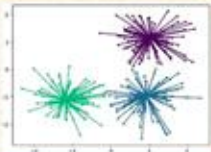
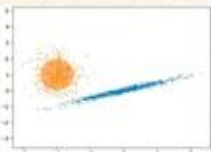
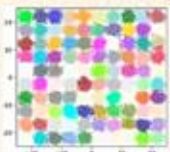
4. Iteration and Refinement

- **Repeat** assignment and update until convergence.
- **Convergence**: Stop when assignments stabilize or iterations reach a limit.

5. Evaluation

- **Evaluate** cluster quality (e.g., silhouette, Calinski-Harabasz, Davies-Bouldin).
- **Visualize** clusters for understanding.

Types of clustering algorithms

Type of Clustering Algorithm	Visual Overview	Description	Algorithm(s)
Centroid-based		Cluster points based on proximity to centroid	KMeans KMeans++ KMedoids
Connectivity-based		Cluster points based on proximity between clusters	Hierarchical Clustering (Agglomerative and Divisive)
Density-based		Cluster points based on their density instead of proximity	DBSCAN OPTICS HDBSCAN
Graph-based		Cluster points based on graph distance	Affinity Propagation Spectral Clustering
Distribution-based		Cluster points based on their likelihood of belonging to the same distribution.	Gaussian Mixture Models
Compression-based		Transform data to a lower dimensional space and then perform clustering	BIRCH

Overview of clustering methods in scikit-learn.

→ <https://scikit-learn.org/1.5/modules/clustering.html>

Example in Scikit-learn.

Comparing different clustering algorithms on toy datasets

→ https://scikit-learn.org/0.18/auto_examples/cluster/plot_cluster_comparison.html

The selection of a suitable clustering algorithm

depends on several factors, including:

- Data characteristics. The shape, size, and distribution of the data.
- Desired number of clusters. The expected or known number of groups.
- Computational resources. The available computational power.
- Interpretability. The ease of understanding and explaining the results.

Centroid-based Clustering

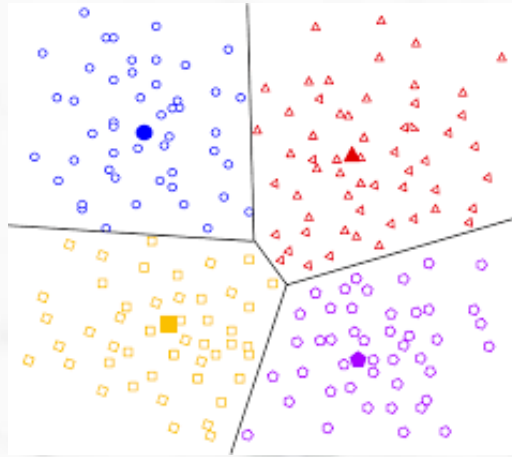
Centroid-based clustering is a method of unsupervised learning where data points are grouped based on their proximity to the centroid of each cluster. A centroid is the average point of all data points within a cluster.

Popular Centroid-Based Algorithms:

- K-means Clustering
- K-medoids Clustering

Key Considerations:

- *Choosing the right value of k.* The number of clusters can significantly impact the results. Techniques like the elbow method or silhouette coefficient can help in determining the optimal k.
- *Handling outliers.* Outliers can influence the centroids and lead to inaccurate clustering. Techniques like outlier detection or robust K-means can be used to address this.
- *Scaling of features.* If features have different scales, it's important to normalize or standardize them to prevent features with larger scales from dominating the clustering process.

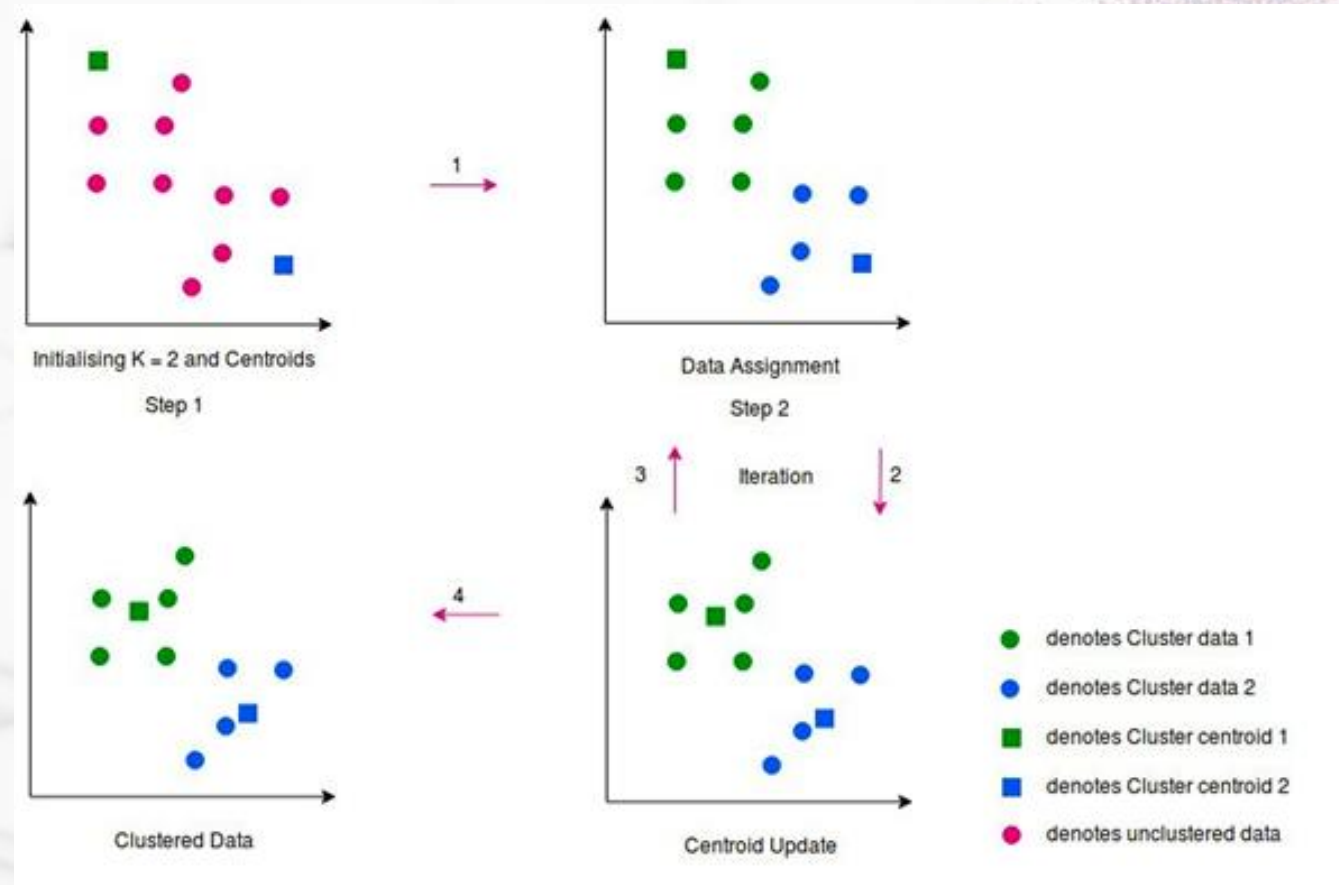
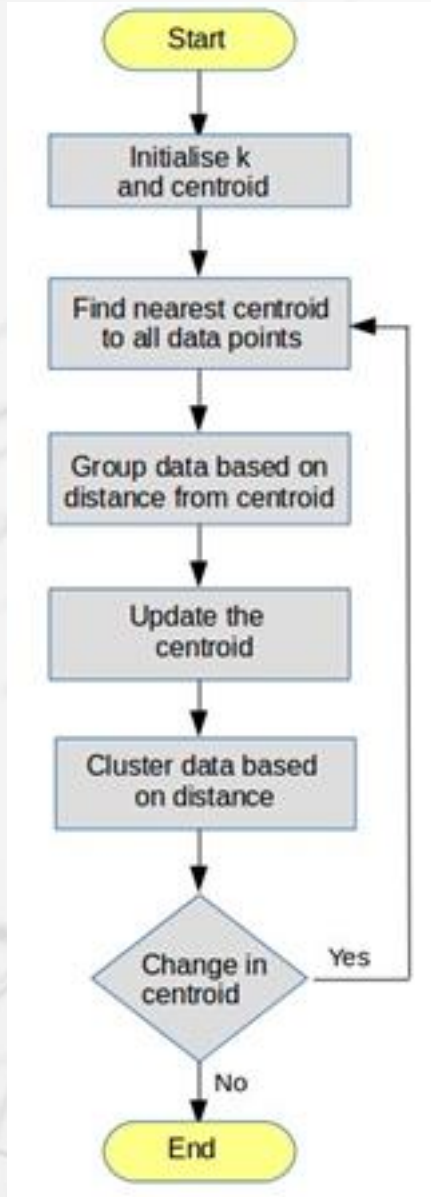


Step-by-Step Guide to Implementing Centroid-based Clustering



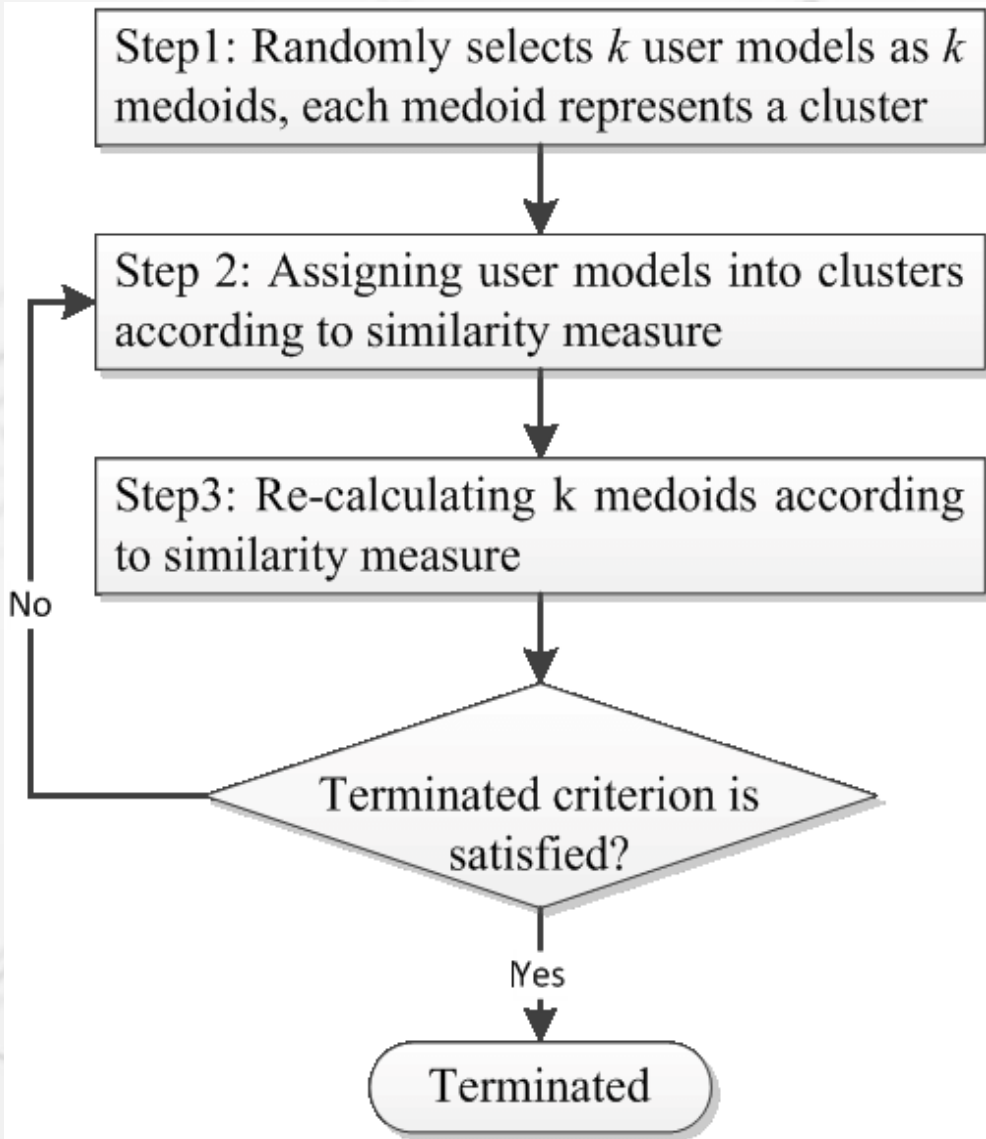
Feature	K-means	K-medoids
Centroid/Medoid	Mean of data points	Data point within the cluster
Update	Recalculate mean	Swap medoids
Sensitivity to outliers	Sensitive	More robust
Efficiency	Faster	Slower
Data type	Numerical	Can handle categorical
When to choose which	If you have a large dataset and need fast results, K-means is a good choice.	If you are concerned about outliers or have categorical data, K-medoids is a better option.

K-means Algorithm



1. Initialize k points, called the 'centroid', randomly or from the data set.
2. Data Assignment Step – Group all data points individually with its nearest centroid to form the clusters.
3. Centroid Update Step – Calculate the value of centroid that is the average of items belonging to each cluster.
4. Reiterate step 2 and 3, till the centroid value remains constant between consecutive iterations.

K-medoids Algorithm



Initialization:

- Randomly select k data points as medoids.

Assignment:

- Assign each data point to the nearest medoid based on a distance metric (e.g., Euclidean distance).

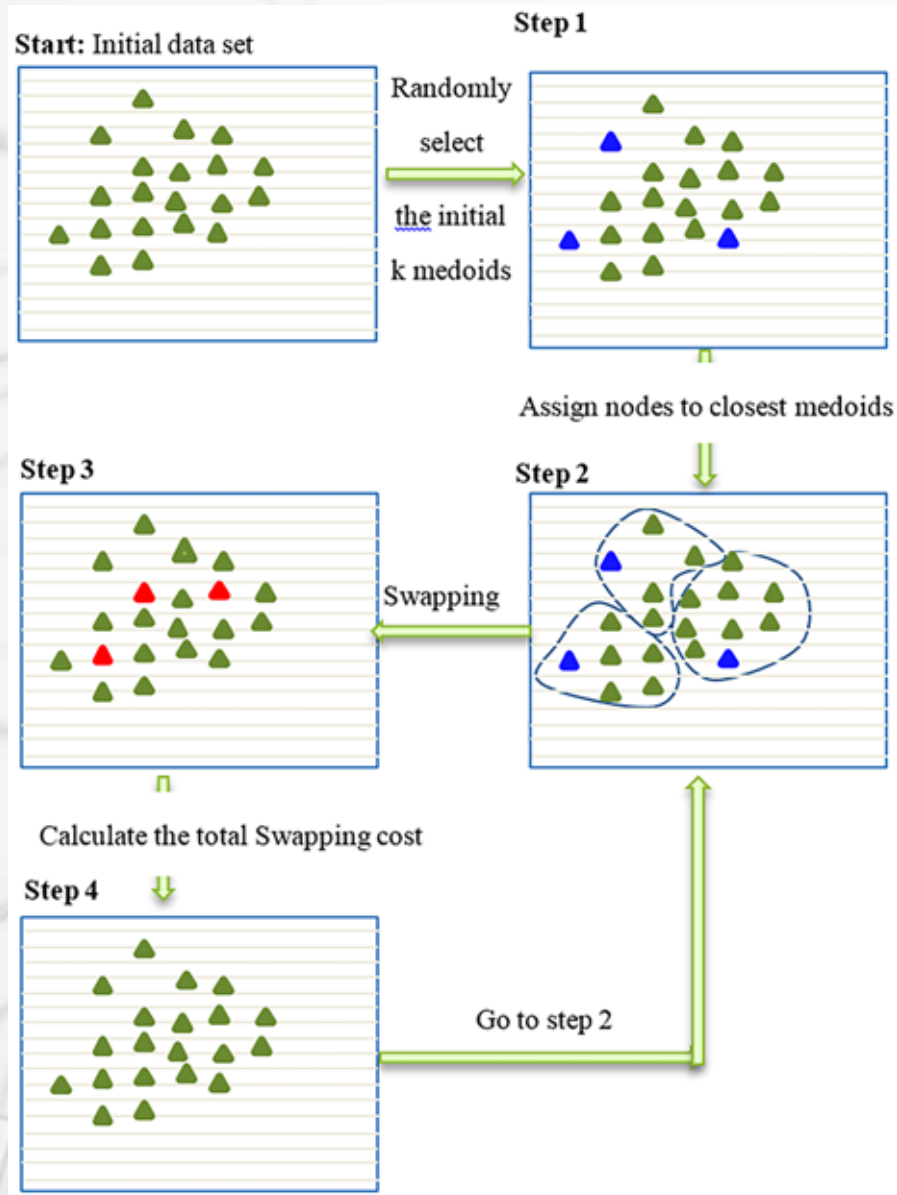
Update:

- For each medoid, calculate the sum of distances to all points assigned to it.
- Swap the current medoid with a non-medoid point if the swap results in a decrease in the overall sum of distances.
- Repeat this process for all medoids.

Repeat:

- Repeat steps 2 and 3 until there are no more changes in the medoid assignments or a specified number of iterations is reached.

K-medoids Algorithm



Example in Scikit-learn:

→ https://codinginfinite.com/k-medoids-clustering-algorithm-with-numerical-example/#google_vignette

Density-based Clustering

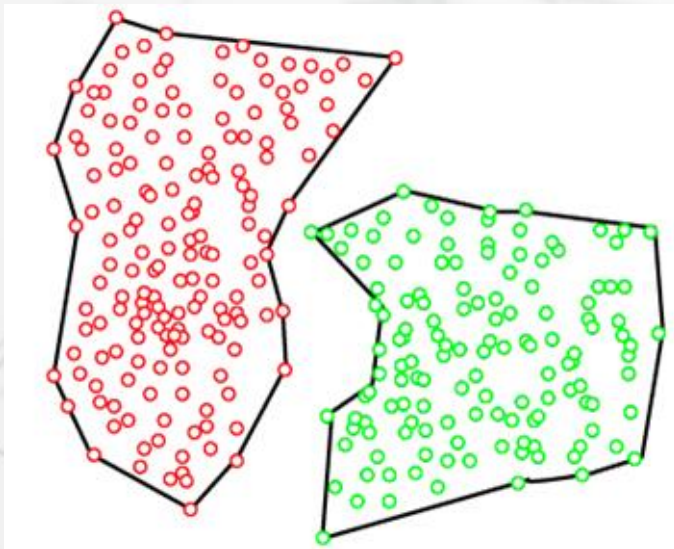
Density-based clustering is a type of unsupervised learning algorithm that groups data points based on their density. It identifies clusters as regions of high density separated by regions of low density. This approach is useful for discovering clusters of arbitrary shapes, unlike centroid-based methods like K-means which assume clusters are spherical.

Popular Density-Based Algorithms:

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
- OPTICS (Ordering Points To Identify the Clustering Structure)

Key Considerations:

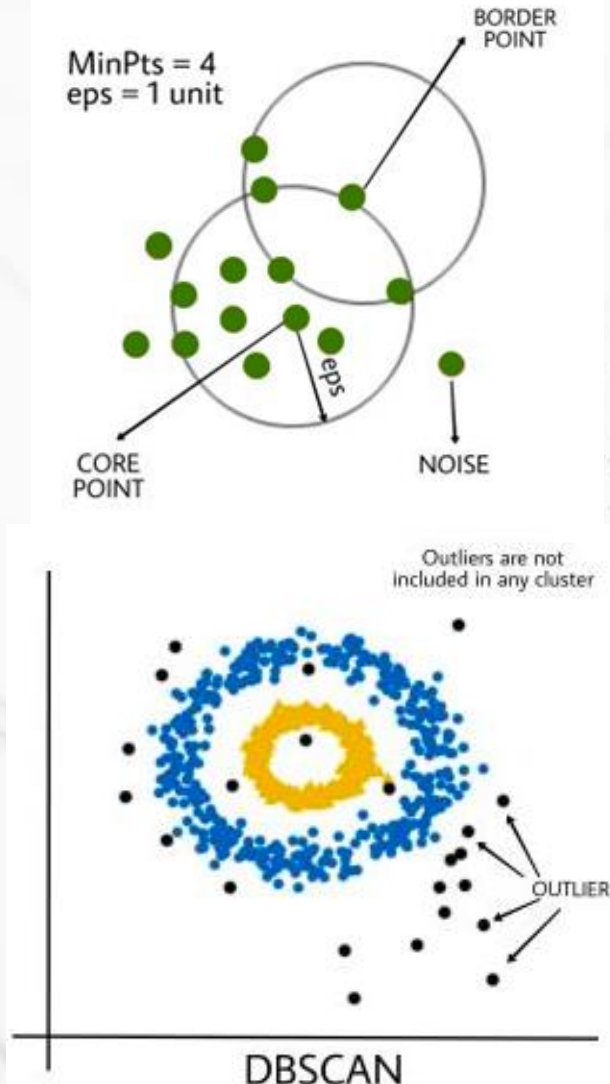
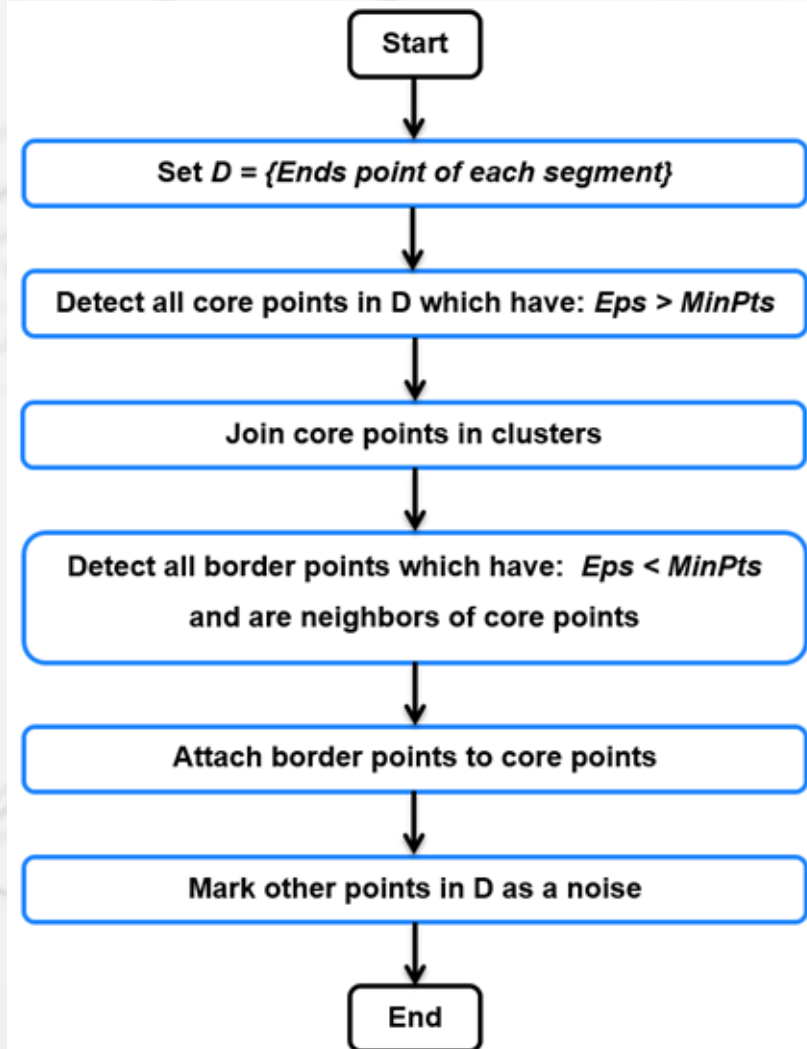
- *Parameter selection.* Choosing appropriate values for ϵ and MinPts in DBSCAN can significantly impact the clustering results.
- *Noise handling.* Density-based methods can effectively handle noise by identifying points that are not part of any cluster.
- *Scalability.* For large datasets, techniques like spatial indexing can improve the efficiency of density-based algorithms.



Feature	DBSCAN	OPTICS
Clustering type	Direct clustering	Hierarchical clustering
Parameters	ϵ and MinPts	MinPts
Cluster structure	Flat	Hierarchical
Parameter sensitivity	Sensitive	Less sensitive
Noise handling	Explicit	Implicit
When to choose which	If you have a clear understanding of the density thresholds in your data and need a flat set of clusters	If you want a hierarchical representation of clusters and are unsure about the optimal density thresholds

DBSCAN Algorithm

DBSCAN is a density-based clustering algorithm that groups data points based on their density. It identifies clusters as regions of high density separated by regions of low density.



Pseudocode For DBSCAN Clustering Algorithm

```
DBSCAN(dataset, eps, MinPts){
  # cluster index
  C = 1
  for each unvisited point p in dataset {
    mark p as visited
    # find neighbors
    Neighbors N = find the neighboring points of p

    if |N| >= MinPts:
      N = N U N'
      if p' is not a member of any cluster:
        add p' to cluster C
  }
```

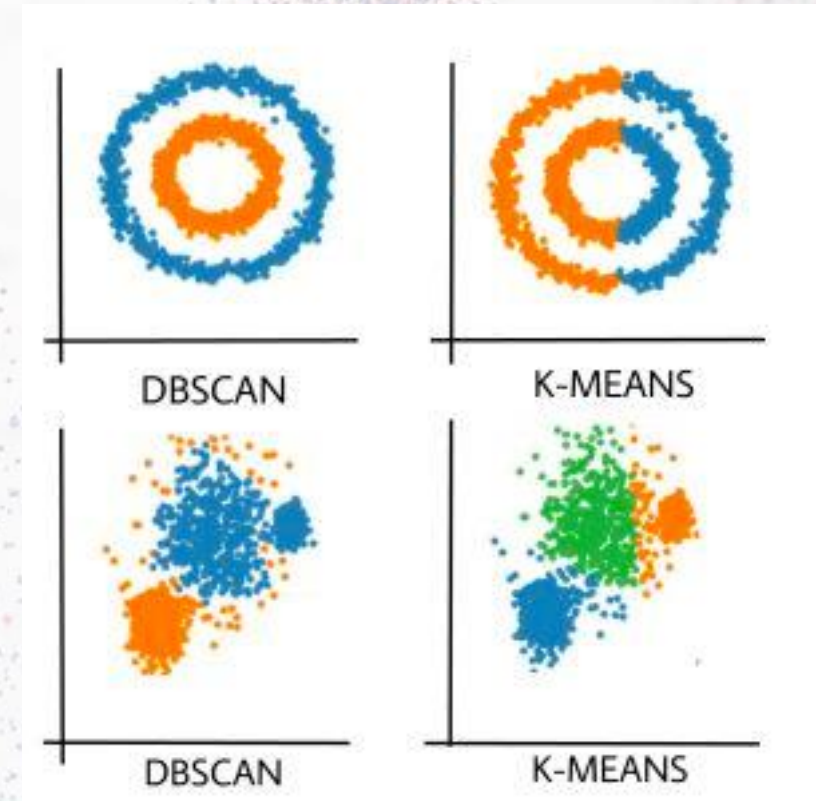
In Scikit_learn,

Difference Between DBSCAN and K-Means

Feature	K-means	DBSCAN
Clustering principle	Centroid-based	Density-based
Cluster shape	Assumes spherical	Handles arbitrary shapes
Number of clusters	Fixed (k)	Determined automatically
Sensitivity to outliers	Sensitive	Robust

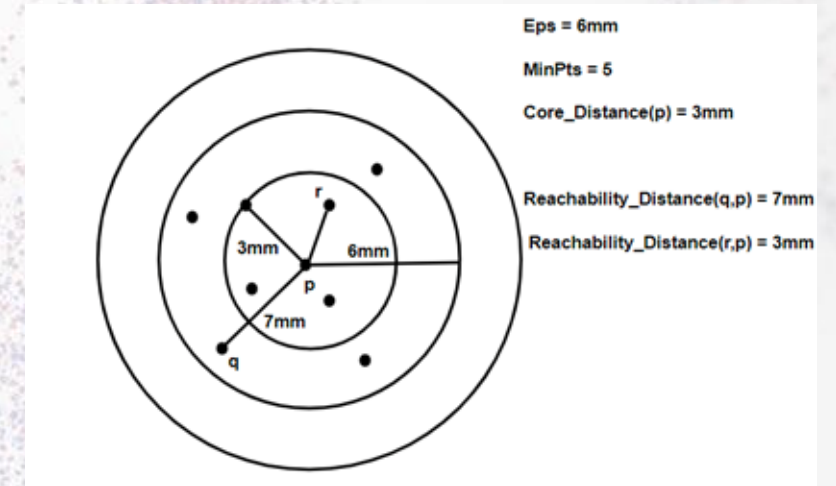
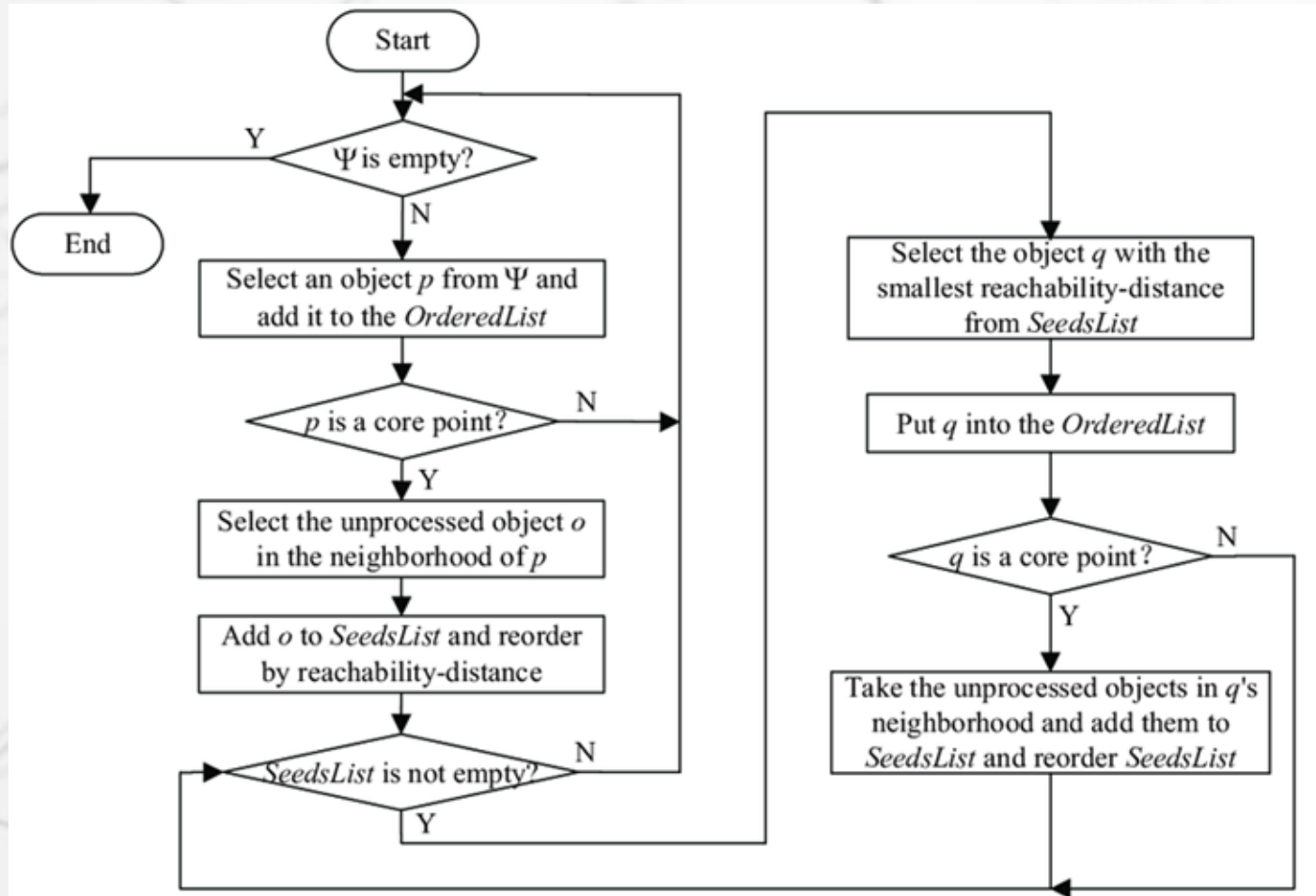
When to choose which:

- **K-means:** If you have a clear understanding of the number of clusters and believe they are roughly spherical, K-means is a good choice.
- **DBSCAN:** If you are unsure about the number of clusters or if they have irregular shapes, DBSCAN is a better option.



OPTICS

OPTICS is a density-based clustering algorithm that extends DBSCAN to create a hierarchical representation of clusters. It is particularly useful when the density of data points varies significantly.



Core distance calculation. For each data point, calculate its core distance, which is the minimum radius such that the point has at least $MinPts$ neighbors within that radius.

Reachability distance calculation. For each data point, calculate its reachability distance from another point. The reachability distance is the maximum of the core distance of the first point and the Euclidean distance between the two points.

Ordering points. Order the data points based on their reachability distance.

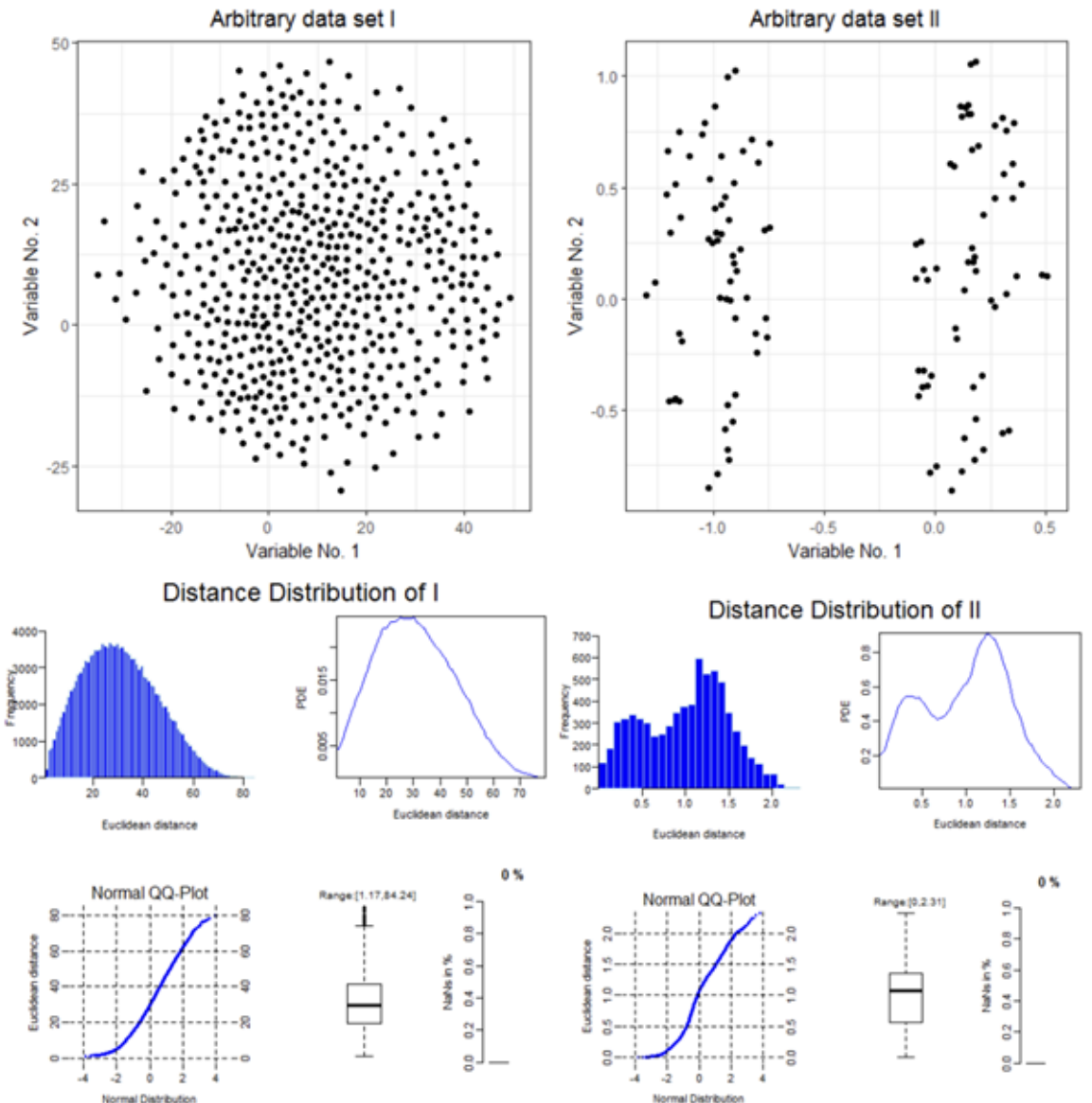
Cluster extraction. Extract clusters from the ordered points based on a desired density threshold.

Distribution-based Clustering

Distribution-based clustering is a type of unsupervised learning algorithm that assumes data points are generated from underlying probability distributions. It aims to group data points that are likely to have been drawn from the same distribution.

Key Considerations:

- *Choosing the number of components.* The number of Gaussian components in GMMs can significantly impact the clustering results. Techniques like the Bayesian Information Criterion (BIC) or Akaike Information Criterion (AIC) can help determine the optimal number.
- *Initialization.* The initialization of GMM parameters can affect the convergence and final results. Techniques like random initialization or k-means clustering can be used.
- *Local optima.* GMMs may converge to local optima, leading to suboptimal solutions. Multiple restarts or other optimization techniques can help mitigate this.



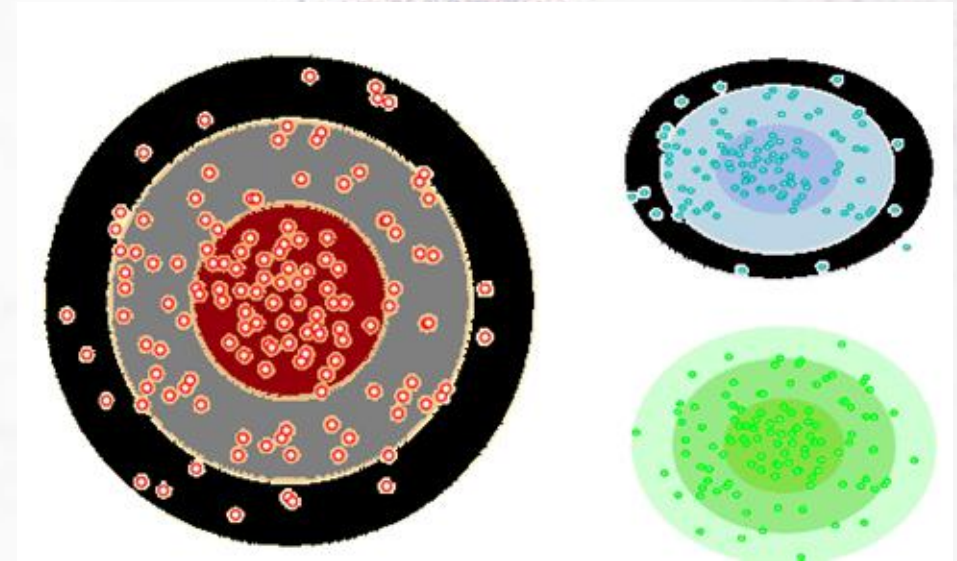
Distribution-based Clustering

Key Distribution-Based Algorithms:

- Gaussian Mixture Models (GMMs)
- Hidden Markov Models (HMMs)
- Mixture Models

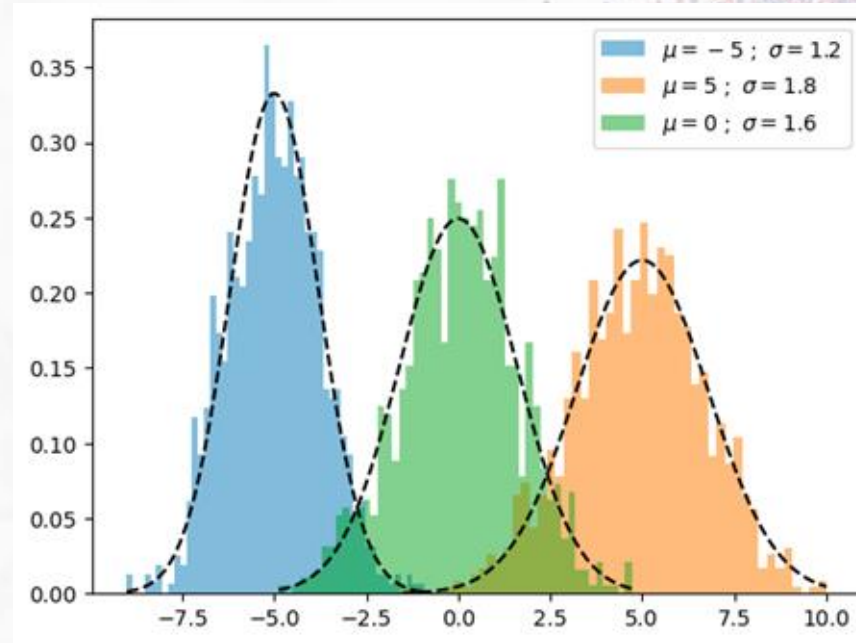
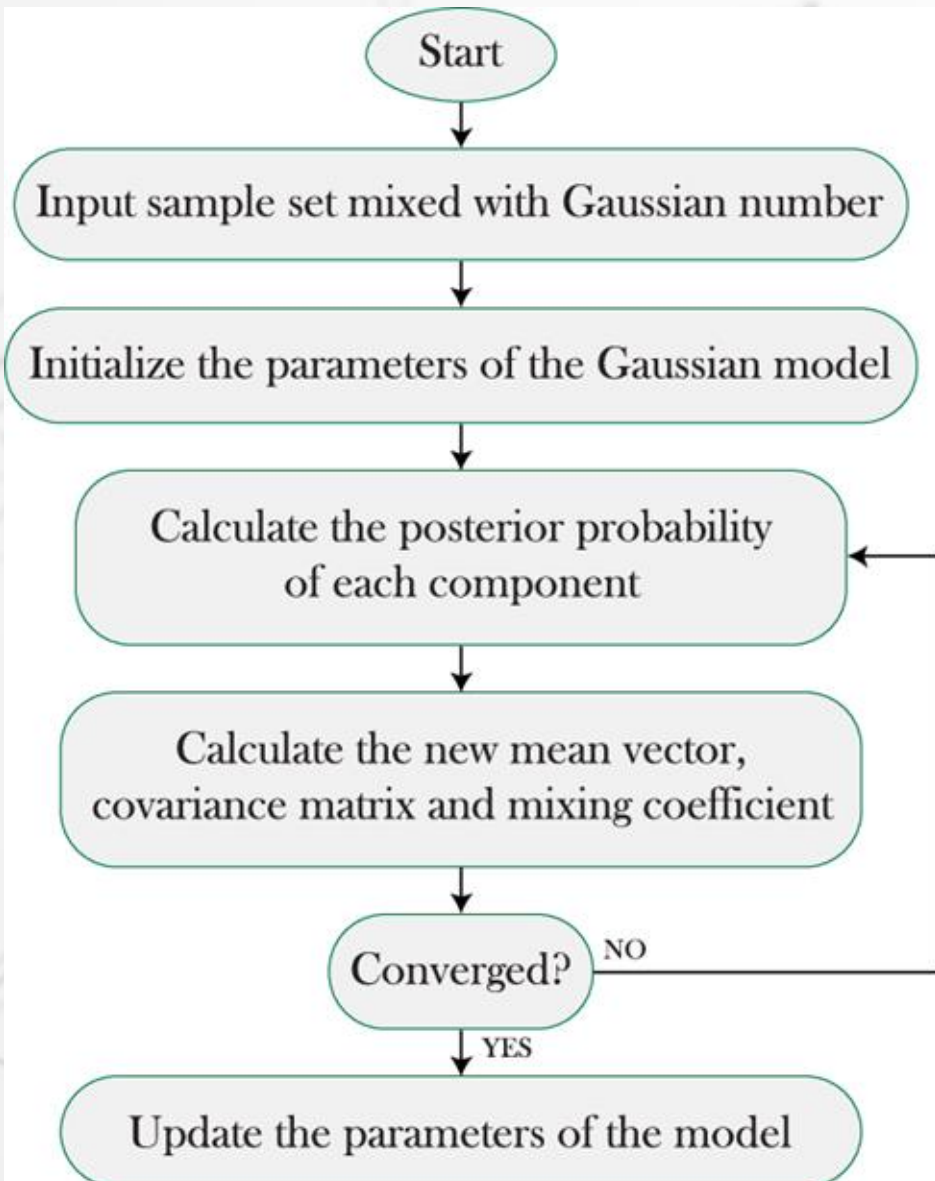
The choice of distribution-based algorithm depends on the specific problem and data characteristics:

- GMMs are suitable for clustering data into overlapping groups.
- HMMs are useful for modeling sequential data with hidden states.
- Mixture Models offer flexibility to model various data distributions, but require careful selection of the appropriate distribution



Feature	GMMs	HMMs	Mixture Models
Data Assumption	Mixture of Gaussian distributions	Markov process with hidden states	Mixture of any parametric distribution
Goal	Identify Gaussian components	Infer hidden states and transition probabilities	Identify underlying distributions and parameters
Method	Expectation-Maximization (EM) algorithm	Baum-Welch algorithm	EM algorithm (or other optimization techniques)

Gaussian Mixture Models (GMMs)



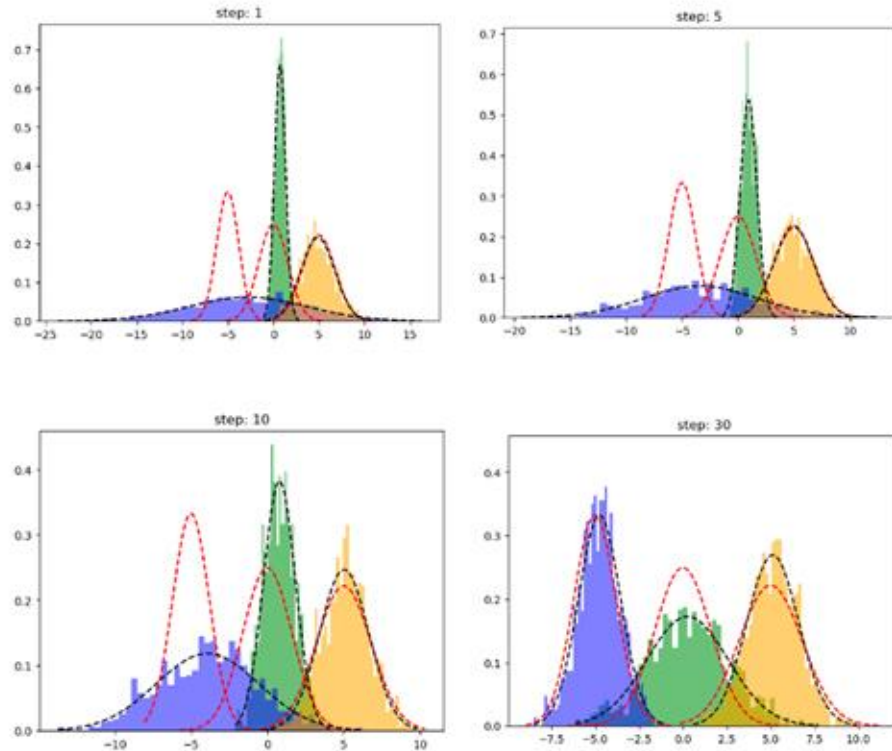
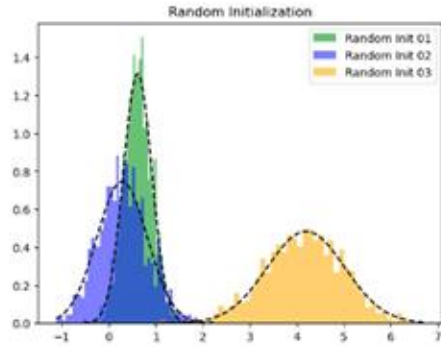
Initialization. Randomly initialize the parameters of each Gaussian component (means, covariances, and weights).

Expectation Step (E-step). Calculate the probability of each data point belonging to each Gaussian component based on the current parameters. This is done using Bayes' theorem.

Maximization Step (M-step). Update the parameters of each Gaussian component to maximize the likelihood of the data given the current assignments. This involves recalculating the means, covariances, and weights using the calculated probabilities.

Repeat. Repeat the E-step and M-step until convergence (no significant change in the parameters).

Gaussian Mixture Models (GMMs)



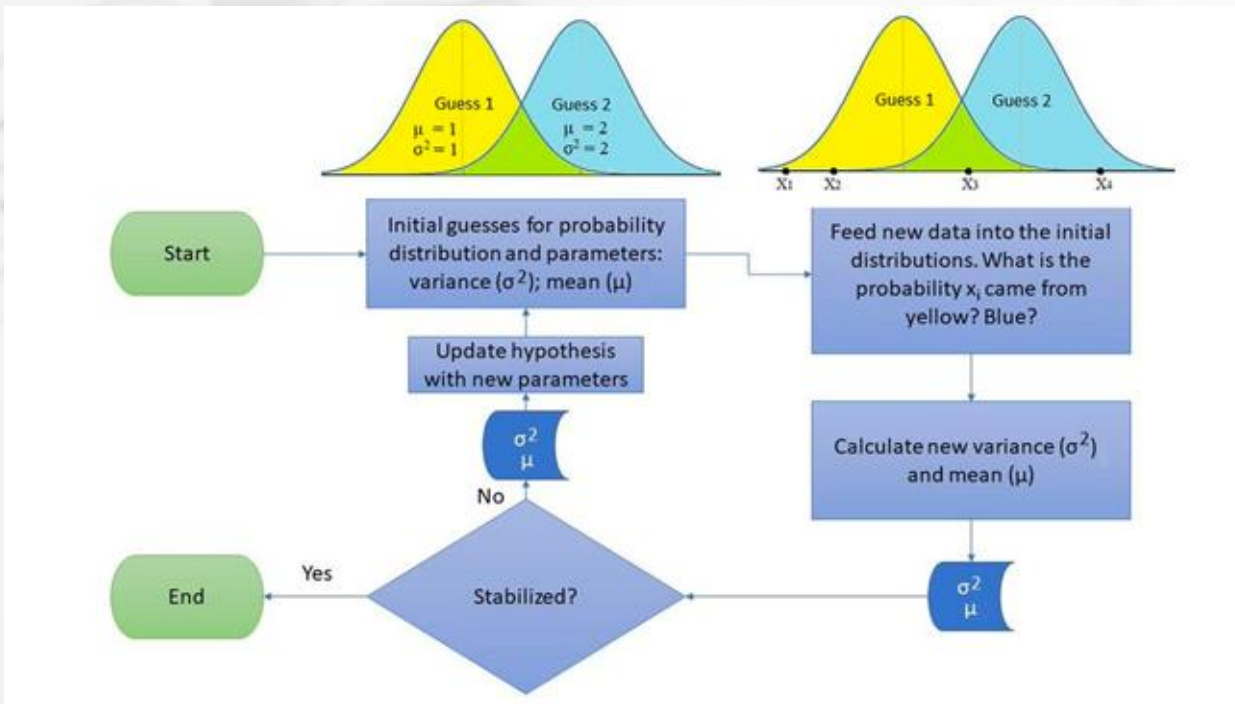
Example in scikit learning:

1) <https://scikit-learn.org/1.5/modules/mixture.html#gaussian-mixture>

2) https://scikit-learn.org/1.5/auto_examples/mixture/plot_gmm_covariances.html#sphx-glz-auto-examples-mixture-plot-gmm-covariances-py

Expectation-Maximization (EM) Algorithm

Expectation-Maximization (EM) is an iterative algorithm commonly used for maximum likelihood estimation in statistical models, especially when there is missing data or latent variables. It's a powerful tool for clustering and other unsupervised learning tasks.



Initialization. Randomly initialize the parameters of the model.

Expectation Step (E-step). Given the current parameter values, calculate the expected values of the missing data or latent variables. This step involves using the current model to estimate the probability distribution of the missing data or latent variables.

Maximization Step (M-step). Given the expected values from the E-step, update the model parameters to maximize the likelihood of the observed data. This involves optimizing the model parameters with respect to the complete data (observed and expected).

Repeat. Repeat the E-step and M-step until convergence (no significant change in the parameters).

Hierarchical Clustering Methods

Hierarchical clustering is a type of unsupervised learning algorithm that creates a hierarchy of clusters, starting from individual data points and merging them into larger clusters until all data points are in a single cluster.

Types of Hierarchical Clustering:

1. Agglomerative (Bottom-Up Approach):

- **Initial Step:** Starts by treating each data point as a separate cluster. So, if there are N data points, you begin with N clusters.
- **Clustering Process:** In each step, the algorithm merges the two clusters that are closest to each other until all the clusters are merged into one big cluster containing all data points.
- **Dendrogram:** The result can be represented in a tree-like structure called a dendrogram, which shows the arrangement of the clusters and their proximity.

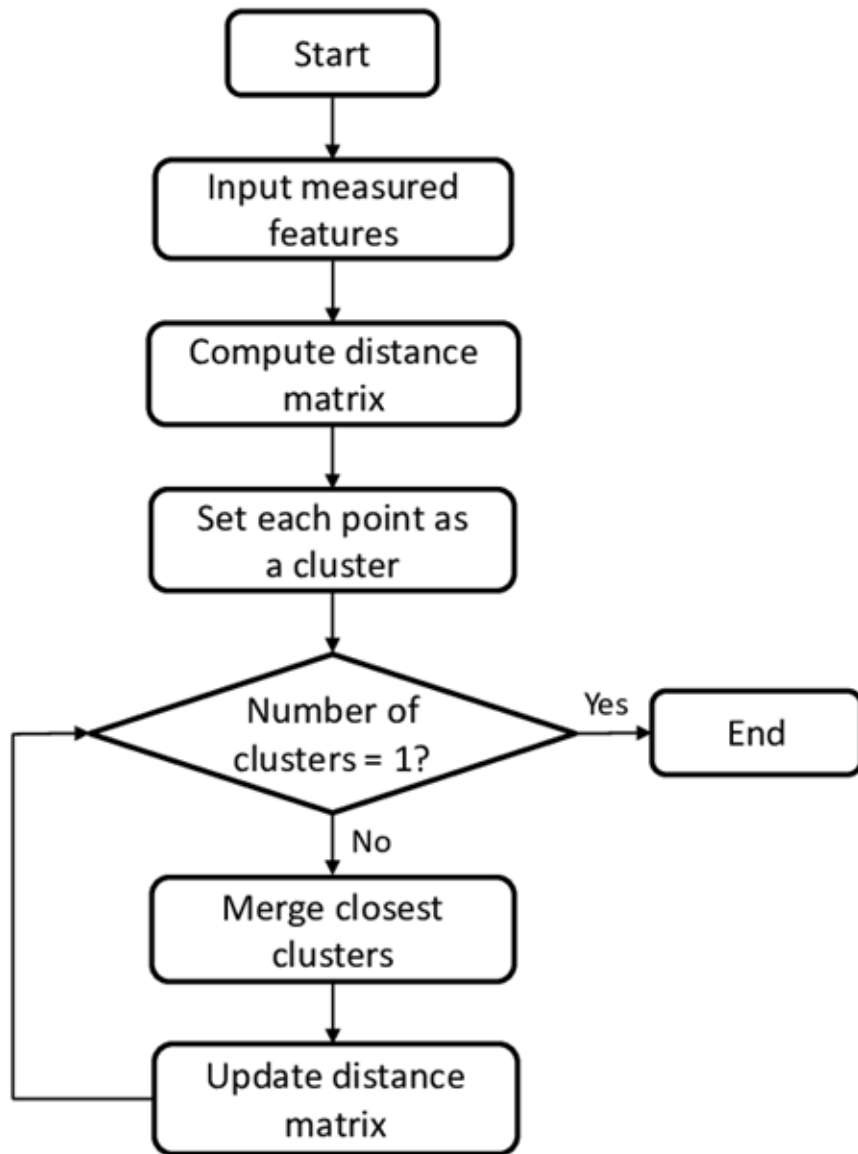
2. Divisive (Top-Down Approach):

- **Initial Step:** Begins with all data points in a single cluster.
- **Clustering Process:** At each step, the algorithm splits the cluster until each cluster contains only one data point.
- **Top-Down Splitting:** This is less common compared to agglomerative clustering and is computationally more intensive.



Feature	Agglomerative Hierarchical Clustering	Divisive Hierarchical Clustering
Approach	Bottom-up	Top-down
Efficiency	Less efficient for large datasets	More efficient for large datasets
Sensitivity to Noise	More sensitive to noise	Less sensitive to noise
Cluster Shape	Can handle clusters of arbitrary shapes	Can handle clusters of arbitrary shapes

Hierarchical Clustering Methods



Hierarchical clustering employs a measure of distance/similarity to create new clusters. Steps for Agglomerative clustering can be summarized as follows:

- Step 1: Compute the proximity matrix using a particular distance metric
- Step 2: Each data point is assigned to a cluster
- Step 3: Merge the clusters based on a metric for the similarity between clusters
- Step 4: Update the distance matrix
- Step 5: Repeat Step 3 and Step 4 until only a single cluster remains

The choice of hierarchical clustering method depends on various factors, including:

- **Dataset size.** For large datasets, divisive methods can be more efficient.
- **Noise level.** For noisy datasets, divisive methods can be more robust.
- **Cluster shape.** Both methods can handle clusters of arbitrary shapes, but the choice of linkage or splitting criteria can influence the results.
- **Desired number of clusters.** Agglomerative methods are more flexible in determining the number of clusters by cutting the dendrogram at different levels.

Hierarchical Clustering Methods

Similarity between Clusters

The main question in hierarchical clustering is how to calculate the distance between clusters and update the proximity matrix. There are many different approaches used to answer that question. Each approach has its advantages and disadvantages.

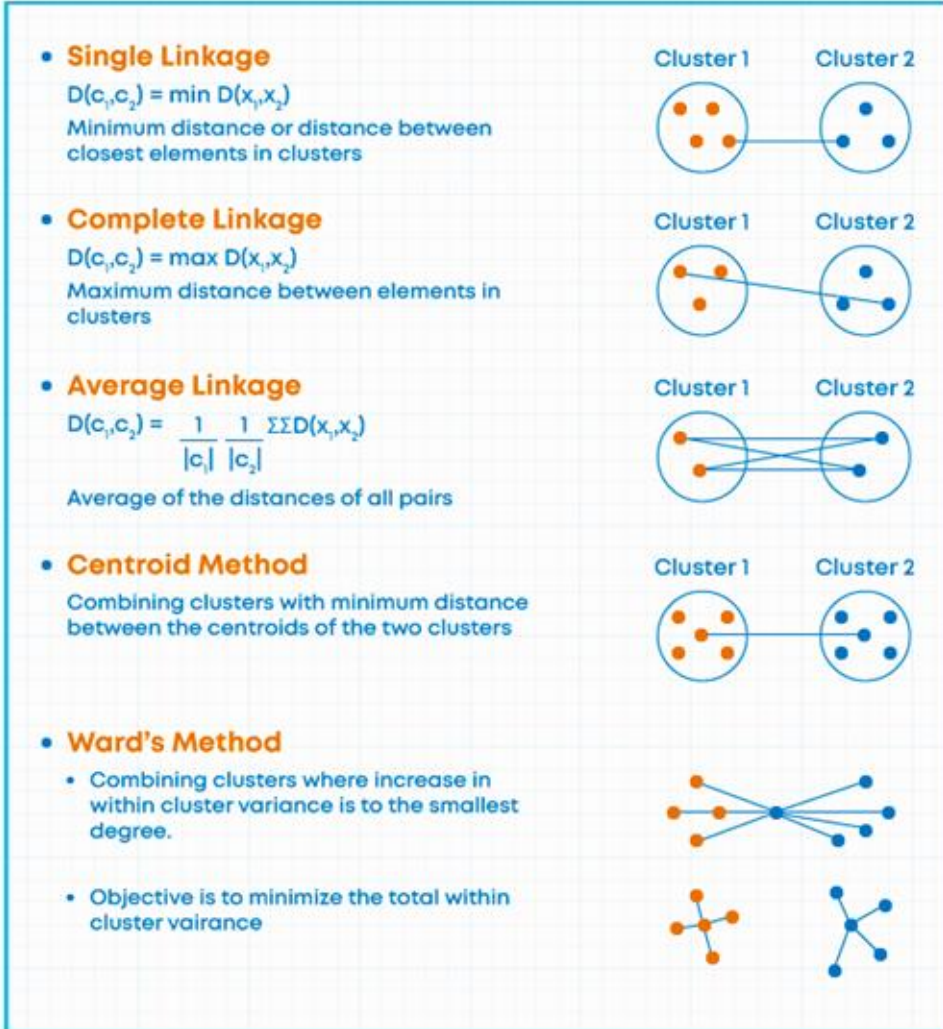
1. Single-Linkage (Minimum Distance) can handle non-spherical clusters, sensitive to outliers, but tends to produce chain-like clusters, sensitive to noise.
2. Complete-Linkage (Maximum Distance). Robust to noise and outliers, but tends to produce tight, spherical clusters, can be sensitive to small clusters.
3. Average-Linkage balances the sensitivity of single-linkage and complete-linkage, but can be computationally expensive for large datasets.
4. Centroid-Linkage relatively efficient, can handle non-spherical clusters, but sensitive to outliers, can produce unbalanced clusters.
5. Ward's Linkage robust to noise and outliers, tends to produce compact, spherical clusters, but can be sensitive to the initial conditions, may not handle non-spherical clusters well.

Choosing the Right Method:

The choice of linkage method depends on the specific characteristics of the dataset and the desired clustering outcome. Consider the following factors:

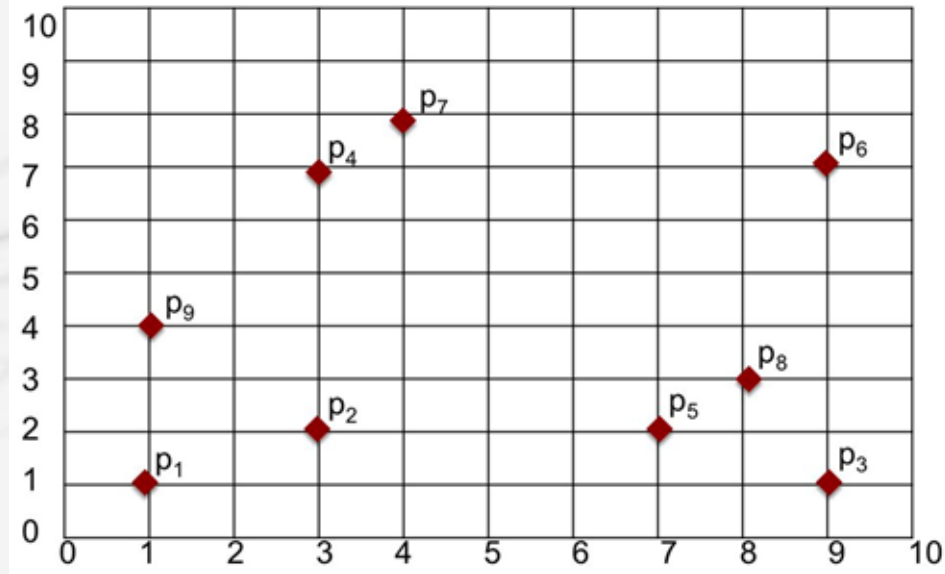
- Noise and Outliers: If the data contains significant noise or outliers, complete-linkage or Ward's method can be more robust.
- Cluster Shape: For non-spherical clusters, single-linkage or average-linkage may be more suitable.
- Computational Cost: For large datasets, centroid-linkage or Ward's method can be more efficient.

Distance Metrics for Hierarchical Clustering:

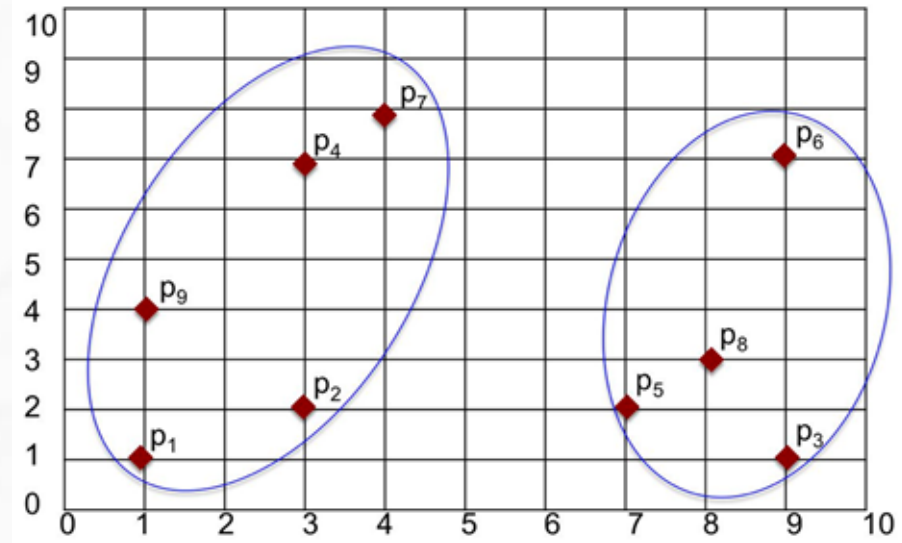


Hierarchical Clustering Methods

Perform a clustering analysis on the provided nine-point, two-dimensional dataset to identify two separate clusters



Point	x	y
p_1	1	1
p_2	3	2
p_3	9	1
p_4	3	7
p_5	7	2
p_6	9	7
p_7	4	8
p_8	8	3
p_9	1	4



Computing a proximity matrix

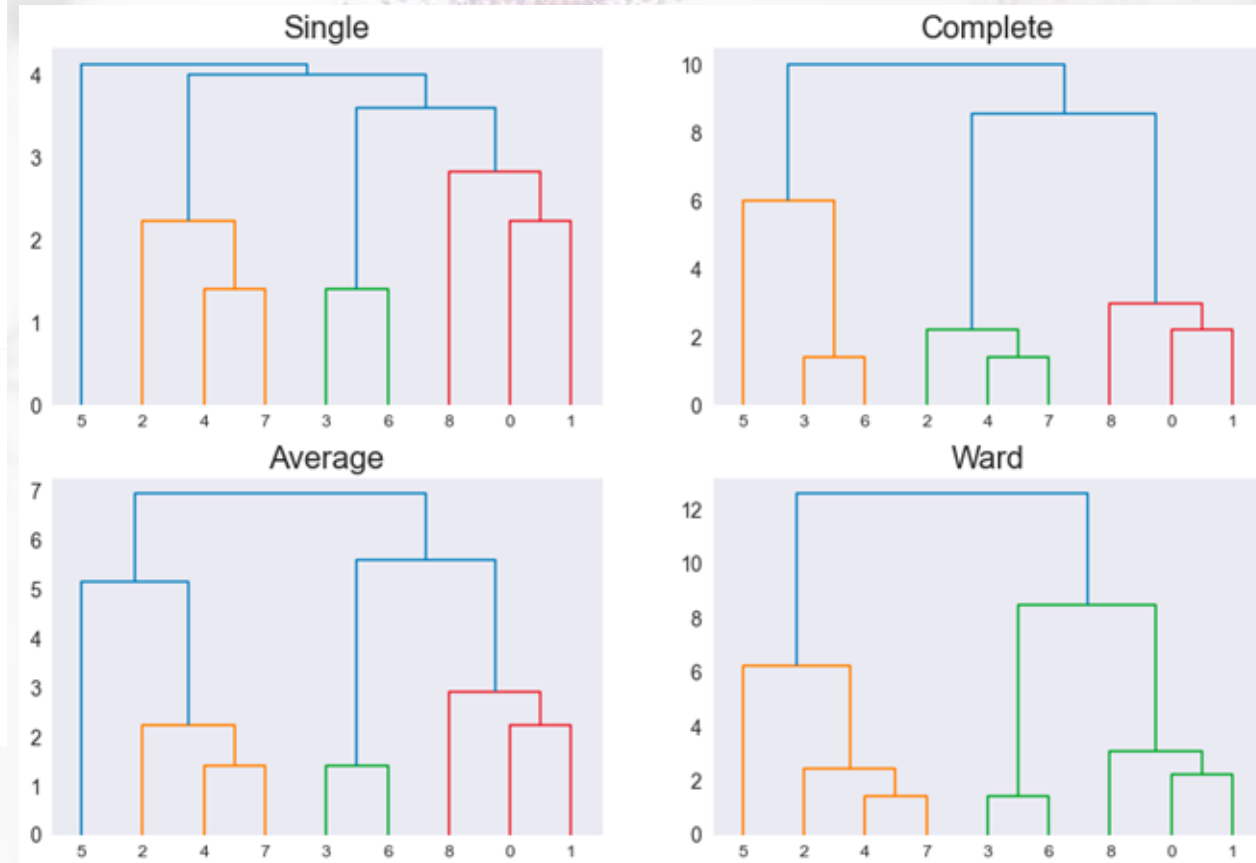
	p_1	p_2	p_3	...	p_n
p_1	$d(p_1, p_1)$	$d(p_1, p_2)$	$d(p_1, p_3)$	...	$d(p_1, p_n)$
p_2	$d(p_2, p_1)$	$d(p_2, p_2)$	$d(p_2, p_3)$	...	$d(p_2, p_n)$
p_3	$d(p_3, p_1)$	$d(p_3, p_2)$	$d(p_3, p_3)$	...	$d(p_3, p_n)$
...	...	...	...	...	...
p_n	$d(p_n, p_1)$	$d(p_n, p_2)$	$d(p_n, p_3)$	...	$d(p_n, p_n)$

Hierarchical Clustering Methods

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

```
Z1 = linkage(X1, method='single', metric='euclidean')  
Z2 = linkage(X1, method='complete', metric='euclidean')  
Z3 = linkage(X1, method='average', metric='euclidean')  
Z4 = linkage(X1, method='ward', metric='euclidean')
```

```
plt.figure(figsize=(15, 10))  
plt.subplot(2,2,1), dendrogram(Z1), plt.title('Single')  
plt.subplot(2,2,2), dendrogram(Z2), plt.title('Complete')  
plt.subplot(2,2,3), dendrogram(Z3), plt.title('Average')  
plt.subplot(2,2,4), dendrogram(Z4), plt.title('Ward')  
plt.show()
```



BIRCH algorithm

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) is a clustering algorithm designed to efficiently handle large datasets. It's particularly useful for exploratory data analysis and preprocessing for other clustering algorithms.

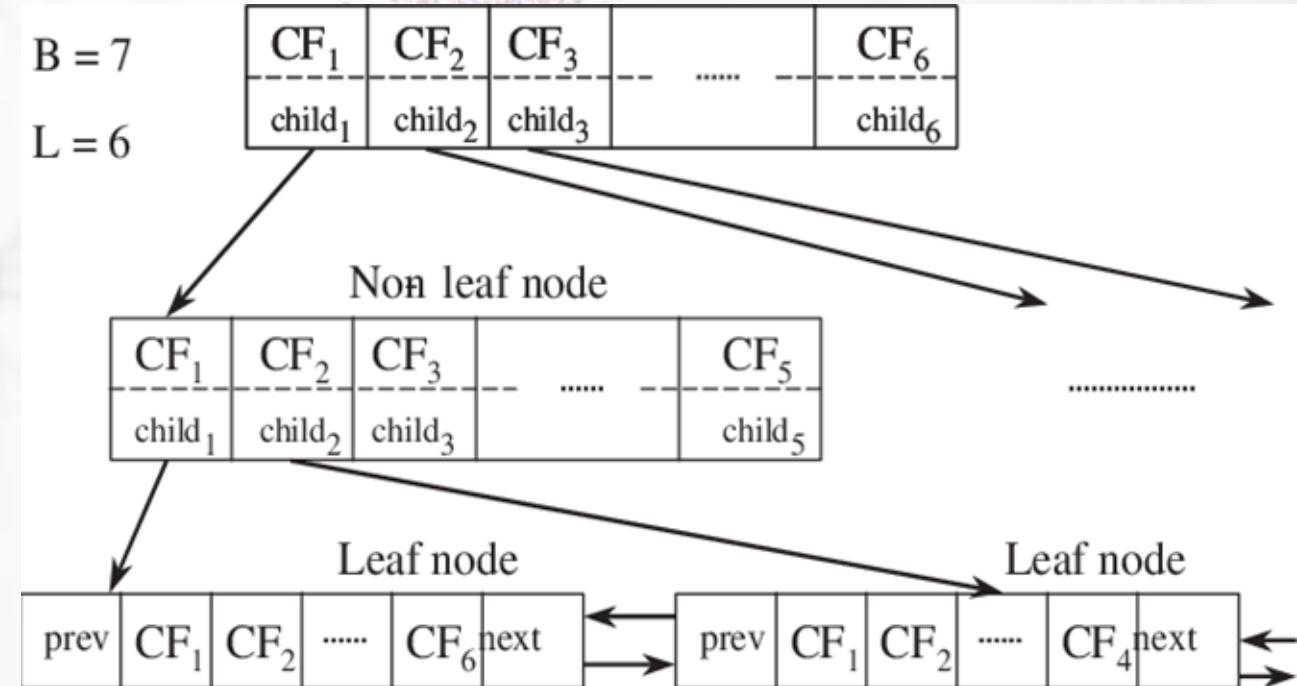
Key Concepts:

- **Clustering Feature (CF):** A CF is a summary of a cluster. It contains three key attributes:

- **Number of points (N):** The number of data points in the cluster.
- **Linear sum (LS):** The sum of all data points in the cluster.
- **Squared sum (SS):** The sum of the squared values of all data points in the cluster.

- **CF Tree:** A tree-based data structure used to organize CFs. It has two types of nodes:

- **Non-leaf nodes:** These nodes have children and store CFs.
- **Leaf nodes:** These nodes store CFs and can have pointers to child nodes.



BIRCH algorithm

How BIRCH Works:

1. Scanning the Database:

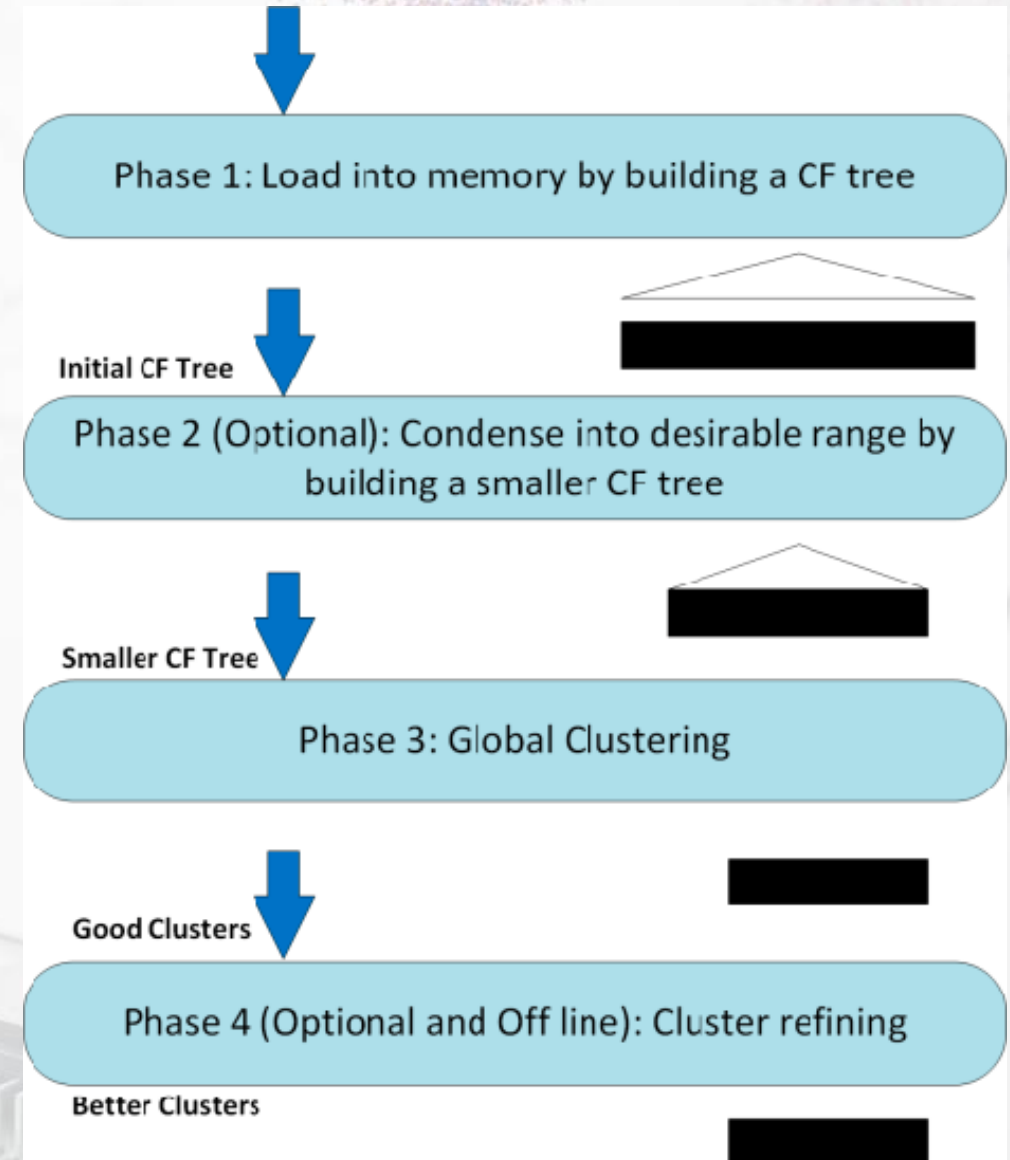
- The algorithm scans the database once, creating CFs for each data point.
- These CFs are inserted into the CF tree.

1. Building the CF Tree:

- The CF tree is built incrementally as data points are processed.
- New CFs are inserted into the tree, and existing CFs are merged if they are close enough.

1. Clustering:

- Once the CF tree is built, clustering can be performed:
 - **Global Clustering:** The CF tree is traversed, and CFs are merged or split to form clusters.
 - **Micro-Clustering:** Smaller clusters can be further refined using other clustering algorithms like K-means or DBSCAN.



BIRCH algorithm

Advantages of BIRCH:

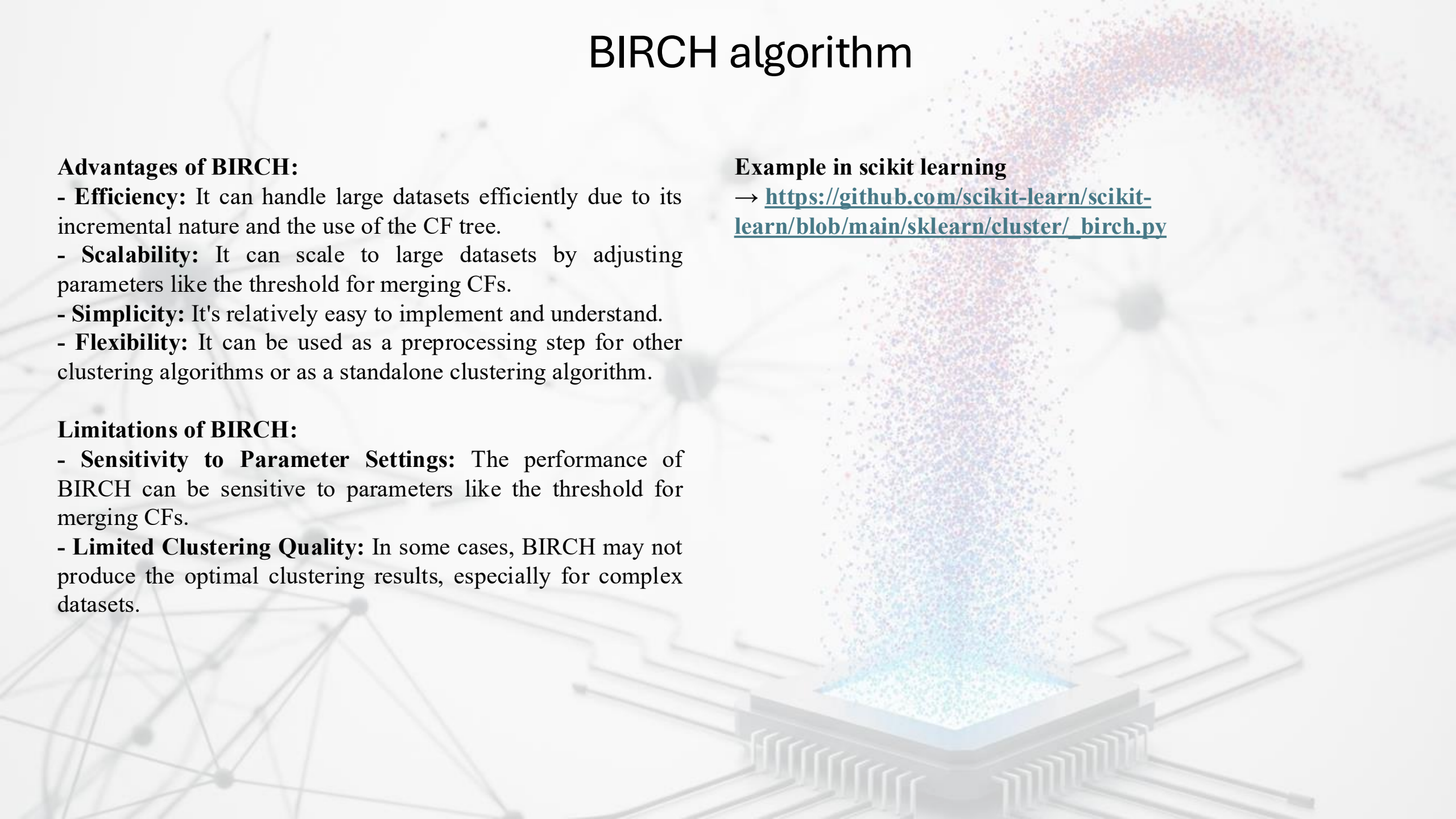
- **Efficiency:** It can handle large datasets efficiently due to its incremental nature and the use of the CF tree.
- **Scalability:** It can scale to large datasets by adjusting parameters like the threshold for merging CFs.
- **Simplicity:** It's relatively easy to implement and understand.
- **Flexibility:** It can be used as a preprocessing step for other clustering algorithms or as a standalone clustering algorithm.

Limitations of BIRCH:

- **Sensitivity to Parameter Settings:** The performance of BIRCH can be sensitive to parameters like the threshold for merging CFs.
- **Limited Clustering Quality:** In some cases, BIRCH may not produce the optimal clustering results, especially for complex datasets.

Example in scikit learning

→ https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/cluster/_birch.py



Measure metrics

Internal Evaluation Metrics

- **Silhouette Coefficient:** Measures how similar a data point is to its own cluster compared to other clusters. A higher coefficient indicates better-defined clusters.
- **Davies-Bouldin Index:** Evaluates the average similarity between each cluster and its most similar cluster. Lower values indicate better-separated clusters.
- **Calinski-Harabasz Index:** Measures the ratio of the sum of between-clusters dispersion and within-cluster dispersion. Higher values indicate better-defined clusters.

External Evaluation Metrics

- **Rand Index:** Measures the similarity between two clusterings. A higher Rand Index indicates better agreement between the two clusterings.
- **Adjusted Rand Index:** A variation of the Rand Index that adjusts for chance agreement.
- **Normalized Mutual Information:** Measures the amount of information shared between two clusterings. Higher values indicate better agreement.

Chỉ số đánh giá cho bài toán Clustering

Chỉ số

Ý nghĩa

Silhouette Score

Đo độ tách biệt và độ gọn của các cụm.
Giá trị càng gần 1 → cụm càng rõ ràng, tách biệt tốt.

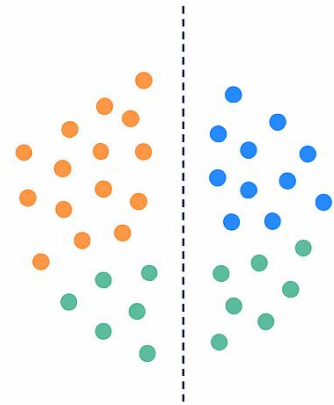
Davies–Bouldin Index (DBI)

Đo mức độ giống nhau giữa các cụm.
DBI càng nhỏ → cụm càng tốt, phân tách rõ hơn.

Calinski–Harabasz Score (CH Score)

Đo mức độ tách biệt giữa các cụm so với độ gọn trong cụm.
CH càng lớn → chất lượng phân cụm càng cao.

Độ tách biệt giữa các cụm



Silhouette Score

$$s = \frac{b - a}{\max(a, b)}$$

- Khoảng cách trung bình đến các điểm cùng cụm
- Khoảng cách trung bình đến các điểm trong cụm gần nhất khác

Davies–Bouldin Index
Mức độ phân tách giữa các cụm

$$DBI = \frac{1}{k} \sum_{j \neq i} \frac{S_i + S_j}{d_{ij}}$$

- DBI càng nhỏ → cụm càng tốt

Calinski–Harabasz Score
Tách biệt / Gọn gàng

$$CH = \frac{\text{Between-cluster variance}}{\text{Within-cluster variance}}$$

- CH càng lớn → cụm càng tách biệt và gọn gàng

Practice Exercise 3

Problem: You have a dataset containing information about customers of a retail store, including their age, gender, annual income, and spending score. Your task is to cluster customers based on their similarities and identify distinct customer segments.

Steps:

1. Data Exploration.

- Analyze the dataset for missing values, outliers, and distributions of features.
- Visualize the data using scatter plots or histograms to identify patterns.

2. Feature Scaling. Normalize or standardize the features (age, annual income, and spending score) to ensure they have a similar scale.

3. Clustering.

- Choose a clustering algorithm (e.g., K-means, K-medoids, DBSCAN, Hierarchical clustering,...).
- Experiment with different parameters and evaluate the results using appropriate metrics.

4. Evaluation.

- Use metrics like silhouette coefficient to assess the quality of the clusters.
- Visualize the clusters using scatter plots or dendrograms to understand the segmentation.

Practice Exercise 3 (additional)

K-Means Image Segmentation Exercise

Problem: Segment a given image into K clusters using the K-means algorithm

Dataset: You can use any image of your choice. For example, a natural image like a landscape or a portrait.

Task:

1. Load the Image:

- Read the image using OpenCV or PIL.
- Convert the image to a suitable color space (e.g., RGB, HSV , or LAB).

2. Preprocess the Image:

- Resize the image to a smaller size for faster processing.
- Flatten the image into a 2D array of pixels, where each pixel is represented as a feature vector (e.g., RGB values).

3. Apply K-Means Clustering:

- Initialize K random centroids, each representing the mean color of a cluster.
- Assign each pixel to the nearest centroid based on Euclidean distance.
- Update the centroids as the mean of the pixels assigned to each cluster.
- Repeat the assignment and update steps until convergence.

4. Segment the Image:

- Replace each pixel with the color of its assigned cluster centroid.
- Reshape the segmented image back to its original dimensions.

5. Visualize the Results:

- Display the original and segmented images side-by-side.

Ref:

<https://medium.com/analytics-vidhya/image-segmentation-using-k-means-clustering-from-scratch-1545c896e38e>

References

<https://www.kaggle.com/code/azminetoushikwasi/different-clustering-techniques-and-algorithms/notebook>

<https://github.com/scikit-learn/scikit-learn/tree/main/sklearn/cluster>

